



All You Need is Representation & Abstraction

By:

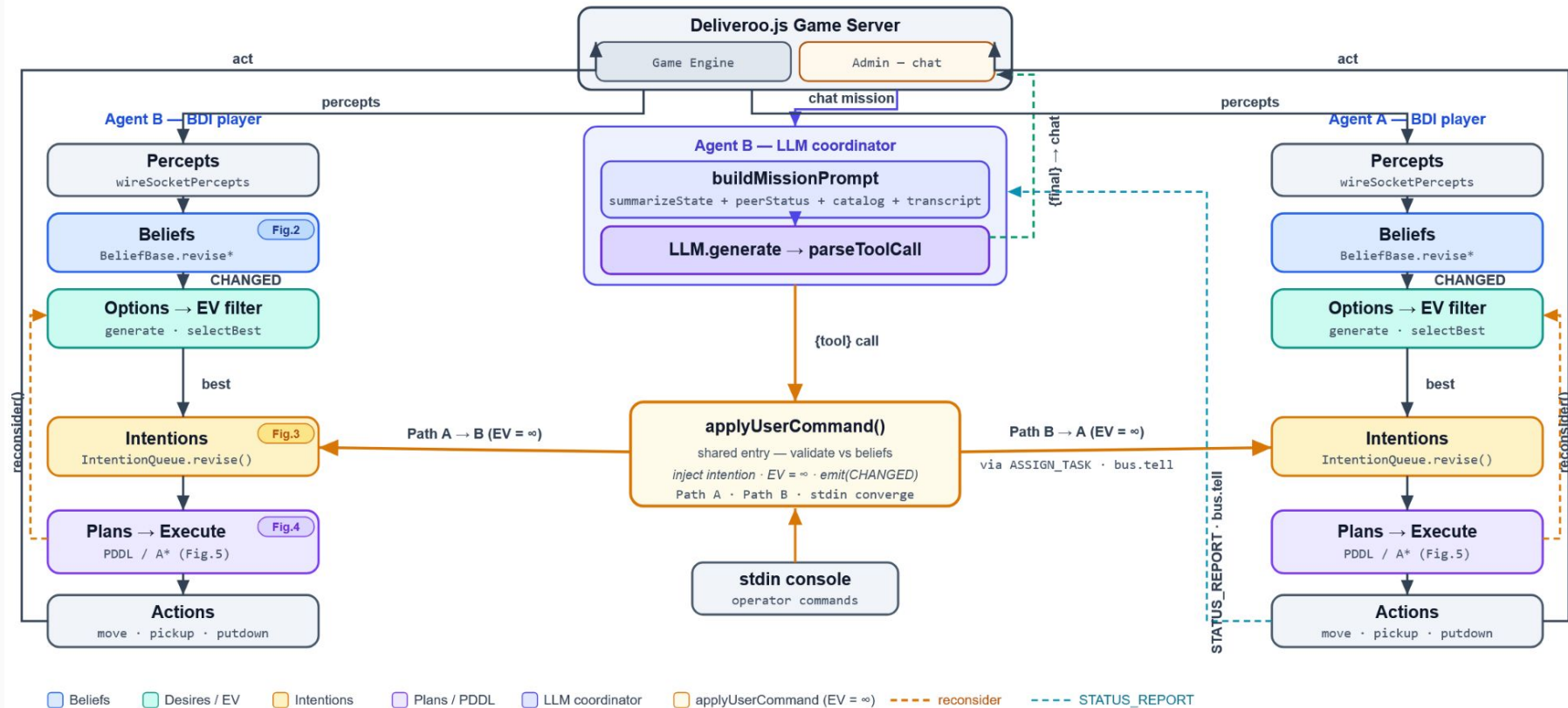
Thomas Gunawisesa (265438)

Stephen Nganga (266153)

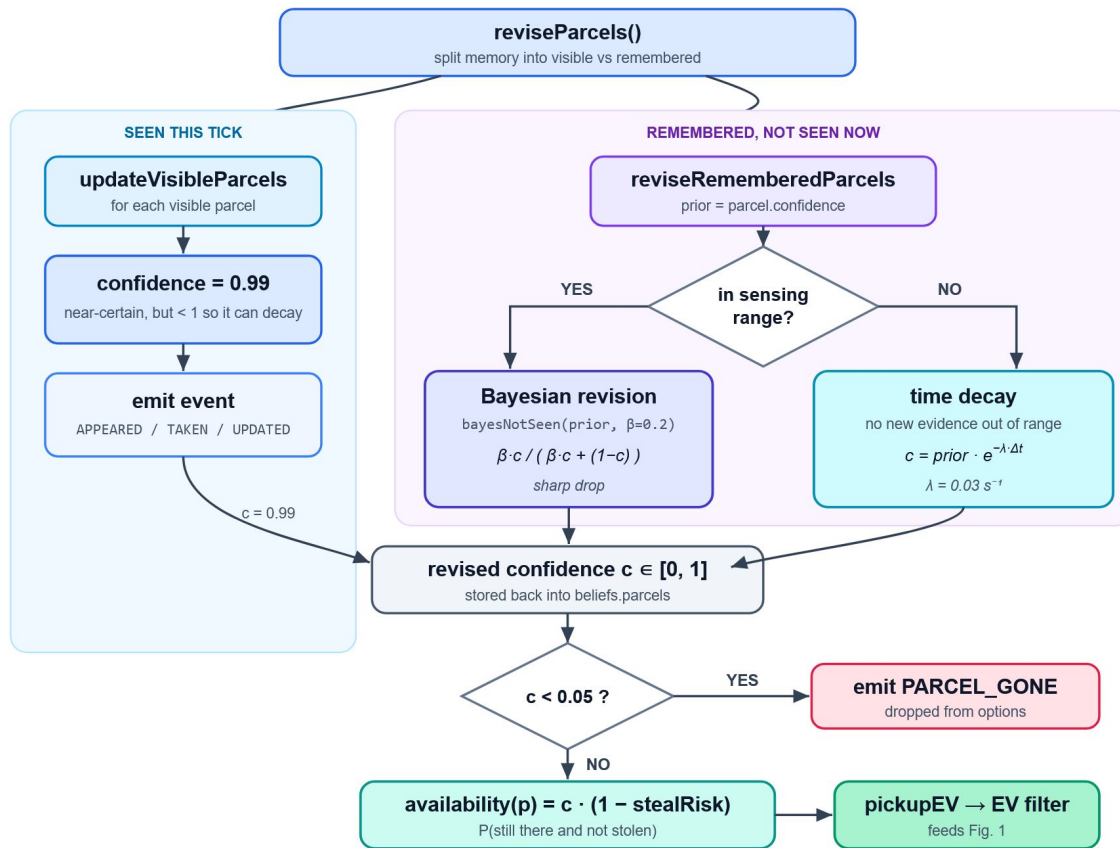
Autonomous Software Agents

2026

Architecture



Belief Revision



Bayesian revision (within range, unseen):

$$P(H|\neg Seen) = \frac{\beta \cdot c}{\beta \cdot c + (1 - c)}$$

where β = sensorMissProbability (0.2), c = prior confidence

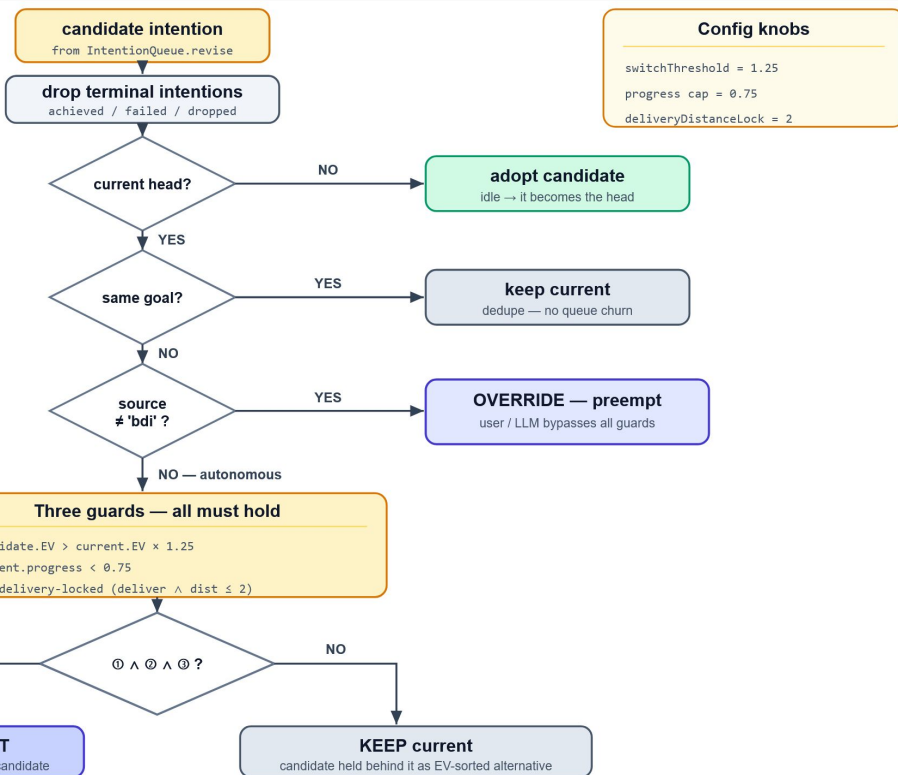
Exponential time decay (out of range):

$$c = \text{prior} \cdot e^{-\lambda \cdot \Delta t}$$

pickupEV = P(available) * decayedReward - travelCost

deliverEV = reward - distance + capacityPressureBonus.

Intention Revision



```
// Candidate from selectBestOption:
{ option: { type:'pickup', parcelId:'p9', target:{x:3,y:7}, reward:30 },
  estimatedEV: 18.5
}

// Current queue before revision:
[ Intention { id:'intention-14',
  predicate: {type:'pickup', parcelId:'p2',target:{x:4,y:5}},
  estimatedEV: 12.0, progress: 0.35, status:'running',
  source:'bdi'
} ]

// Guard checks:
//   muchBetter:      18.5 > 12.0 * 1.25 = 15.0  -> true
//   lowSunkCost:      0.35 < 0.75                -> true
//   deliveryLocked:   type != 'deliver'          -> false
//   => PREEMPT

// Queue after revision:
[ Intention { predicate: {type:'pickup', parcelId:'p9',...},
  estimatedEV:18.5, status:'pending', source:'bdi' },
  Intention { predicate: {type:'pickup', parcelId:'p2',...},
  estimatedEV:12.0, status:'dropped', source:'bdi' }
]

// User override case:
{ option:{type: 'move', target:{x:10, y:5}},
  estimatedEV: Infinity,
  source: 'user'
}

//   source != 'bdi' => bypass all guards, preempt immediately
```

Plan Execution

PlanSelector.select() — first applicable wins

GoPickUp

GoDeliver

GoExplore

AStarMove

BlindMove

GoPickUp

1 · reach parcel

2 · AStarMove ▶ core loop

3 · pickup() · reviseParcels()

GoDeliver

1 · retarget nearest zone

2 · AStarMove ▶ core loop

3 · putdown() · clear beliefs

GoExplore

1 · patrol spawners (round-robin)

2 · AStarMove ▶ core loop

3 · resume-on-block

AStarMove.execute() — per-step loop

compute path

planner.plan → PDDL, else A*

path found?

NO

fail 'no-path'

non-recoverable → drop

YES

▶ for each step in path

done

all steps done → success

completeCurrent()

blocked or
move failed?

YES

markBlocked · replan

recoverable → keep intention

NO

advance

reviseYou · updateProgress(done,total)

reconsider:
head changed?

YES

throw 'reconsidered'

deliberate() picks the new head

NO → next step

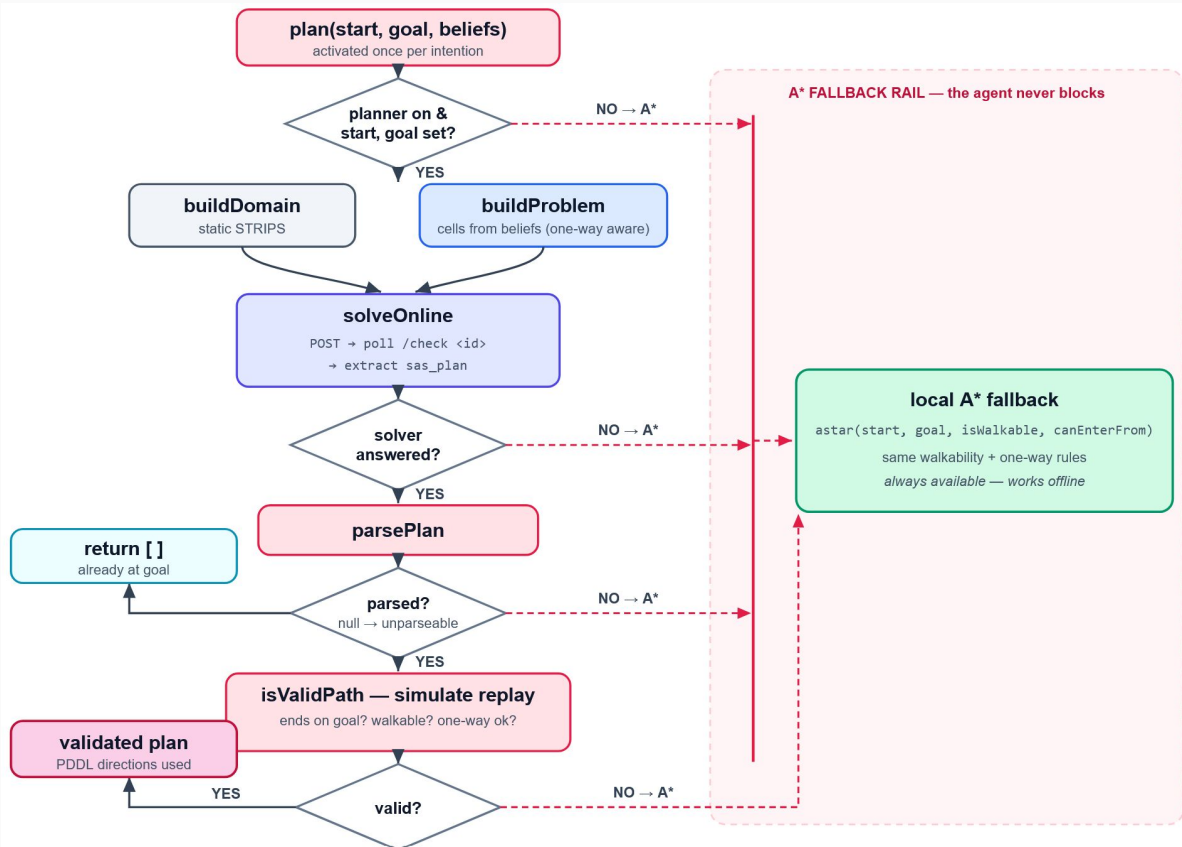
```
// PlanSelector tries in order:
//   GoPickUp.isApplicable({predicate:{type:'pickup',...}}) -> true => SELECTED

// AStarMove computes path:
astar({x:0,y:0}, {x:3,y:7}, isWalkable, canEnterFrom)
// -> ['right','right','right','up','up','up','up']

// Per-step context:
{ beliefs: BeliefBase{ me:{x:1,y:0} },
  actions: { move: (dir) => socket.emitMove(dir),
             pickup: () => socket.emitPickup() },
  planner: ExternalPlanner{...},
  reconsider: () => { const curr = deliberate();
                    return !curr.samePredicate(activeIntention)
                  }
}

// After step 2, reconsider() returns true (new parcel p20 spawned nearby, EV=40):
//   -> throw PlanFailure('reconsidered', true)
//   -> BDI loop catches, continues to next cycle with p20
```

PDDL Integration



```

// buildProblem output:
"(define (problem deliveroo-nav)
 (:domain deliveroo-grid)
 (:objects c0_0 c1_0 c2_0 c3_7 - cell right left up down - direction)
 (:init (at c0_0)
 (move-dir c0_0 c1_0 right)
 (move-dir c1_0 c0_0 left)
 (move-dir c1_0 c2_0 right)
 (move-dir c2_0 c3_0 right))
 (:goal (at c3_7)))"

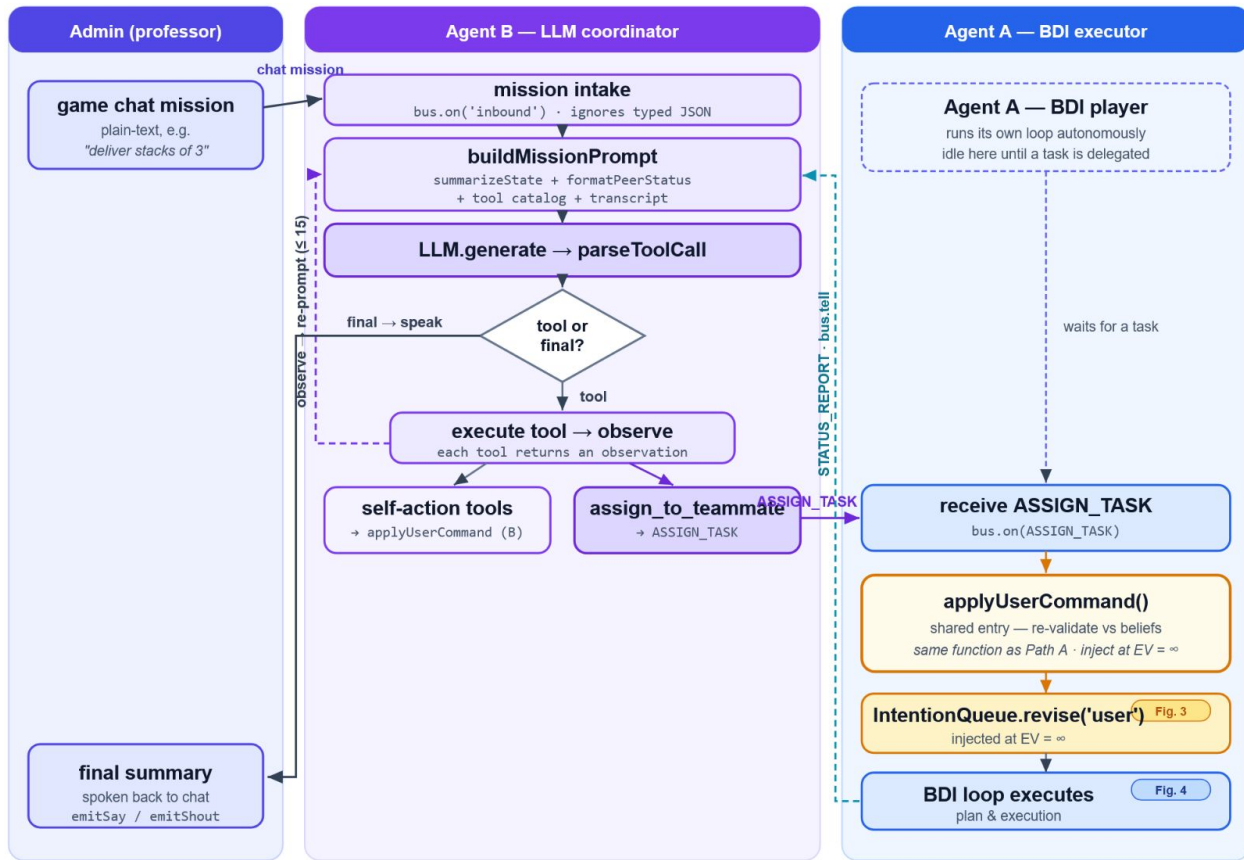
// solveOnline response (after polling):
{ result: { output: { sas_plan: "(move c0_0 c1_0 right)\n..." } } }

// parsePlan output:
['right', 'right', 'right', 'up', 'up', 'up', 'up']

// isValidPath: simulate from (0,0), verify each tile isWalkable + canEnterFrom
// -> passes => return directions

// Timeout case (3s):
// astarPath already computed => return astarPath immediately
    
```

Coordination



```

// Mission intake — plain chat detected (no protocol field):
bus.on('inbound', ({ from, name, payload }) => {
  if (!payload || !payload.protocol) return // skip typed coordination
  const text = payload.text // "tell your teammate to pause"
  runMission(text, { from, name })
})

// Turn 1 — LLM observes:
{ tool: 'observe', args: {} }
// -> summarizeState: "me: (15,1) score=70\ncarrying: 0\n...\npaused: false"
// -> formatPeerStatus: "teammate BDI (f95f32) at (18,0) carrying 1, current
// intention {\`type\`:\`pickup\`,...}"

// Turn 2 — LLM delegates pause to A:
{ tool: 'assign_to_teammate', args: { type: 'set_policy', policy: { paused:
true } } }
// -> validateCommand -> bus.tell('f95f32', ASSIGN_TASK, {
//   targetId: 'f95f32',
//   command: { type: 'set_policy', policy: { paused: true } }
// })
// -> A receives, applyUserCommand:
// { applied: true, reason: 'policy-updated' }
// -> A's strategy.paused = true
// -> A's deliberate() returns current intention (frozen)
// -> A's runBdiLoop() returns early (paused guard)

// A's next STATUS_REPORT (now shows paused):
{ agentId: 'f95f32', name: 'BDI', x: 18, y: 0, carrying: ['p7730'],
intention: null, paused: true }

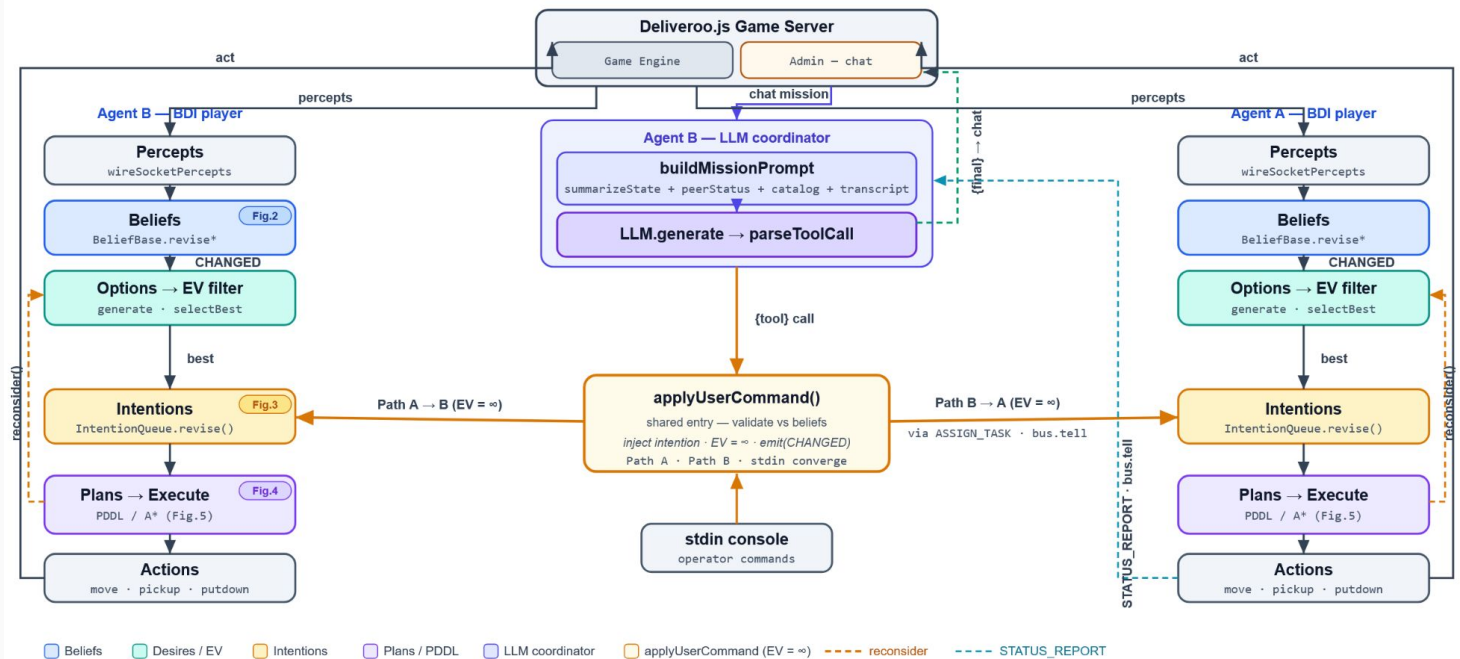
// Turn 3 — LLM sees A is paused, finishes:
{ final: 'Paused teammate Agent A. He was carrying 1 parcel at (18,0).' }
// -> emitSay('admin', final) -> spoken in game chat

// Later — "tell teammate to continue":
// LLM sees formatPeerStatus: "teammate BDI (f95f32) at (18,0) carrying 1,
// current intention none (PAUSED)"

// Turn 1:
{ tool: 'assign_to_teammate', args: { type: 'set_policy', policy: { paused:
false } } }
// -> A receives, applies, strategy.paused = false
// -> A's STATUS_REPORT: paused: false

// Turn 2:
{ final: 'Resumed teammate Agent A. He is now moving again.' }
  
```


Conclusion



Thank you!